Part 3: Offline password cracking and password KDFs

Estimated time: 40 minutes. Environment: course Docker container.

Ethical and authorized use: use only the fictional wordlist, CSV files, and local accounts supplied for this lab. Do not use leaked passwords or third-party verifiers. All guessing in Exercise A is local and offline.

Part 3 connects attack cost to defensive password storage. First, compare dictionary guessing against fast unsalted and salted SHA-256 records. Then benchmark deliberately slower password KDFs and use a local verifier-only login database. The goal is to understand what salt changes, what it does not change, and why password storage also needs an adjustable work factor.

Learning objectives

After completing this part, you should be able to:

- implement transparent offline dictionary attacks using hashlib and csv;
- reuse a candidate digest across unsalted accounts;
- explain why a unique salt requires per-account candidate computation;
- measure and compare hash-operation counts and elapsed time;
- distinguish a fast general-purpose hash from a password KDF;
- identify the principal parameters of PBKDF2, bcrypt, scrypt, and Argon2id;
- report a median benchmark with its configuration and hardware context; and
- inspect a stored authentication record and confirm it contains no plaintext.

Exercise A: fast-hash dictionary attacks

Assume an authorized tester has already received two small synthetic password databases. No login server is contacted, so lockout, network latency, and server-side rate limiting do not apply.

Data model and attack logic

../data/unsalted_hashes.csv has this schema:

```
account,sha256
```

For each word, compute SHA256(UTF8(word)) once and compare the digest with all target rows.

../data/salted_hashes.csv has this schema:

```
account,salt_hex,sha256
```

For each account and word, decode the public salt and compute SHA256(salt || UTF8(word)). A digest calculated with one account's salt cannot be reused for an account with a different salt.

```
Unsalted work: approximately number_of_words hash operations
Salted work:   up to number_of_accounts * number_of_words operations
```

The exact count can be lower if checking stops after a candidate is found. State your stopping rule when interpreting the measurements.

Tasks

1. Inspect the wordlist and both CSV schemas without changing target values.
2. Implement crack() in crack_unsalted.py using one hash per candidate.
3. Implement crack() in crack_salted.py, decoding each hexadecimal salt.
4. Return (recovered, computation_count, elapsed_seconds) from both functions.
5. Run compare_cost.py and compare computation counts as well as wall time.
6. Explain how each method scales to more accounts and a larger dictionary.
7. Relate the offline behavior to the observable SSH requests from Part 2.

Inspect the local inputs:

```
cd /workspace/labs/lab01/part3_password_kdfs
head ../data/lab01-small.txt
sed -n '1,5p' ../data/unsalted_hashes.csv
sed -n '1,5p' ../data/salted_hashes.csv
```

'crack_unsalted.py' usage

This student starter accepts the unsalted database followed by the wordlist:

```
python3 crack_unsalted.py ../data/unsalted_hashes.csv ../data/lab01-small.txt
```

Before implementation it stops at NotImplementedError. After completing crack(), output should have this shape:

```
({'fictional_account': 'recovered_candidate'}, HASH_COUNT, ELAPSED_SECONDS)
```

Hash each candidate once and use the result as a lookup against all targets.

‘crack_salted.py‘ usage

```
python3 crack_salted.py ../data/salted_hashes.csv ../data/lab01-small.txt
```

It returns the same tuple shape. Compute a separate digest for each candidate/account-salt pair. Do not hard-code account names, candidates, counts, or digests in either starter.

‘compare_cost.py‘ usage

After both TODO implementations work, run:

```
python3 compare_cost.py ../data
```

Expected output format; values depend on the implementation and machine:

```
experiment   found   hashes    seconds     hashes/s
unsalted         N        N   0.000000            N
salted           N        N   0.000000            N
```

‘test_dictionary_cost.py‘ usage

Run fixture checks before implementation:

```
python3 -m unittest test_dictionary_cost.py -v
```

After completing both crack() functions, enable implementation checks:

```
LAB01_GRADE=1 python3 -m unittest test_dictionary_cost.py -v
```

The tests use temporary fictional inputs and do not reveal the supplied database candidates.

Exercise B: slow password hashing

Fast SHA-256 makes Exercise A inexpensive. A password KDF intentionally raises the cost of every legitimate verification and every attacker guess. Modern designs may impose both CPU and memory cost. Salt still provides uniqueness; the work factor provides deliberate cost. Neither mechanism creates entropy for a weak password.

Mechanisms being compared

Mechanism	Main classroom parameter	Intended observation
SHA-256	none	Very fast; unsafe baseline for passwords
PBKDF2-HMAC-SHA-256	iterations	Repeats a pseudorandom function
bcrypt	logarithmic cost	Purpose-built and deliberately CPU-expensive
scrypt	n, r, p	Adds configurable memory cost
Argon2id	time, memory, parallelism	Modern memory-hard password hashing

The benchmark uses one deterministic salt only to make timing repeatable. Real registration must generate a fresh random salt per password, as register_login_demo.py does. Production parameters must be tuned on the deployment hardware and threat model.

Tasks

1. Run the three-trial benchmark and record every parameter and median time.
2. Repeat with five trials and compare stability and ordering.
3. Explain why the fastest result is not best for password storage.
4. Register a fictional local account and inspect auth_db.json.
5. Verify once with the chosen fictional password and once with another input.
6. Identify the algorithm, iterations, salt, and verifier stored in JSON.
7. Relate increased legitimate-login cost to increased offline-guessing cost.

‘benchmark_kdfs.py‘ usage

```
python3 benchmark_kdfs.py --trials 3
```

Output has this shape; timings will differ:

```
SHA-256 (unsafe)             ... ms
PBKDF2-100k                  ... ms
bcrypt-cost-10               ... ms
scrypt-N=2^14                ... ms
Argon2id-32MiB               ... ms
Results are machine-specific; tune parameters for each deployment.
```

Repeat with more trials:

```
python3 benchmark_kdfs.py --trials 5
```

The script performs one warm-up before collecting samples and reports their median. Record the algorithm parameters and container host context with results.

‘register_login_demo.py‘ usage

Use a fictional password that you can enter again, but do not place it on the command line:

```
python3 register_login_demo.py register student01
Password:
registered

python3 register_login_demo.py verify student01
Password:
verified
```

A different input should produce authentication failed. Inspect the generated database:

```
python3 -m json.tool auth_db.json
```

Confirm that it stores algorithm metadata, iterations, salt, and verifier, but no plaintext password. auth_db.json is ignored by Git. To isolate an experiment, choose a file under /tmp:

```
python3 register_login_demo.py register student02 --db /tmp/lab01-auth.json
python3 register_login_demo.py verify student02 --db /tmp/lab01-auth.json
```

These commands remain interactive; do not put passwords in shell arguments.

Completion criteria

You have completed Part 3 when both cracking starters run without NotImplementedError, the comparison prints two rows, all enabled tests pass, the benchmark reports all five mechanisms, local registration and verification work, and no plaintext password appears in JSON. Report computation counts, timings, KDF parameters, hardware context, and explanations-but do not publish

recovered candidates or authentication passwords.

Checkpoint questions

1. Why can one unsalted candidate digest serve all accounts?
2. Why must records with distinct salts be checked separately?
3. What protection does salt provide, and what does it not provide?
4. Why is SHA-256 timing a useful baseline but unsuitable for password storage?
5. How does raising a work factor affect legitimate verification and offline guessing?
6. Which demonstrated password KDFs are designed to consume significant memory?
7. Why must KDF parameters be tuned on deployment hardware?
8. Why is an offline attacker unaffected by server-side login rate limiting?